

Machine Learning Project

NASA Space Mission Classification AI

Implementation Track — Machine Learning with
RandomForestClassifier



OVERVIEW

Project Roadmap

A structured walkthrough of the complete ML classification pipeline — from problem definition to future improvements.

01



Problem Statement & Objective

02



Dataset Overview

03



Feature Description

04



Data Preprocessing Pipeline

05



Model Architecture — RandomForest

06



Implementation & Code

07



Results & Accuracy Analysis

08



Confusion Matrix

09



Key Takeaways

10



Future Work



Problem Statement & Objective

GOAL

Build an AI classifier to categorize NASA space missions into their correct mission types based on mission attributes.

- Classify missions into types: **Flyby, Orbiter, Lander, Rover, Observatory, Communications**
- Leverage a synthetic dataset with realistic NASA mission features
- Apply supervised learning using scikit-learn `RandomForestClassifier`
- Achieve high accuracy for reliable mission type prediction



Conceptual classification pipeline

Synthetic Dataset Overview

Sample rows from the NASA space mission training dataset

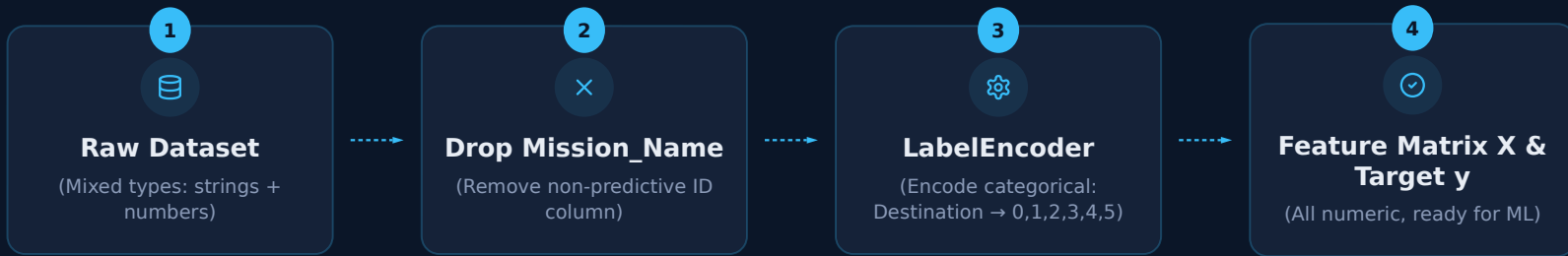
	Mission_Name	Destination	Duration_Days	Crew_Size	Budget_M	Success_Rate	Mission_Type
1	Voyager_X		4,380	0	865	0.92	
2	Artemis_IV		14	4	2,200	0.98	
3	Hubble_Next		7,300	0	1,500	0.95	
4	Mars_Scout		687	0	450	0.88	
5	ISS_Relay		3,650	0	320	0.99	
Synthetic dataset with 6 mission types and 200 samples							

Feature Engineering & Description

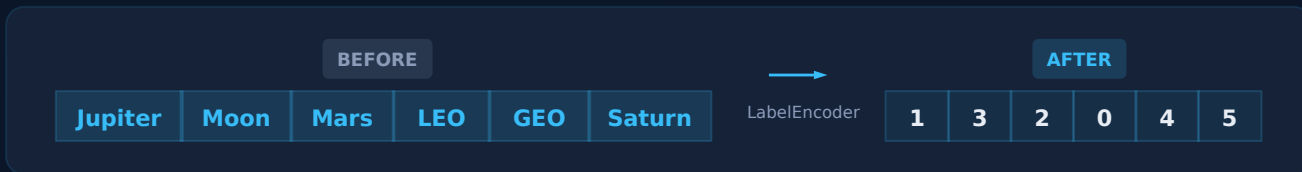
FEATURE NAME	DESCRIPTION / TYPE
1.Mission_Name 	Unique identifier for each mission
2.Destination 	
3.Duration_Days 	Total mission duration in days
4.Crew_Size 	Number of crew members (0 for unmanned)
5.Budget_Million 	Estimated budget in millions USD
6.Success_Rate 	Historical success probability (0.0 - 1.0)
7.Mission_Type 	



Data Preprocessing Pipeline



⚡ Encoding Example — Destination Feature



Each unique string value is mapped to an integer (alphabetical order: GEO=0, Jupiter=1, LEO=2, Mars=3, Moon=4, Saturn=5)



Preprocessing — Code Snippet

preprocess.py

```
1import pandas as pd
2from sklearn.preprocessing import LabelEncoder
3from sklearn.model_selection import train_test_split
4
5# Load synthetic dataset
6df = pd.read_csv('nasa_missions.csv')
7
8# Drop non-predictive column
9df = df.drop('Mission_Name', axis=1)
10
11# Encode categorical features
12le = LabelEncoder()
13for col in df.select_dtypes(include='object').columns:
14    df[col] = le.fit_transform(df[col])
15
16# Split features and target
17X = df.drop('Mission_Type', axis=1)
18y = df['Mission_Type']
19
20# Train-test split (80/20)
21X_train, X_test, y_train, y_test = train_test_split(X,
```

💡 LABELENCODER

Converts each unique string to an integer (0 to n-1), making categorical data compatible with ML algorithms.

📊 TRAIN/TEST SPLIT

80% of data for training, 20% for testing. `random_state=42` ensures reproducible results.

✕ DROP ID COLUMN

`Mission_Name` is a unique identifier with no predictive value — removed before training.

RandomForest Classifier — How It Works



⚙️ Hyperparameters

`n_estimators = 100`

`criterion = gini`

`bootstrap = True`

`random_state = 42`

💡 Key Concepts

- **Bootstrap:** Each tree trains on a random sample drawn with replacement
- **Gini Impurity:** Measures how often a random element would be misclassified
- **Ensemble:** Combines 100 weak learners into one strong classifier
- **Voting:** Final class = most-predicted class across all trees



Model Training — Code Snippet

RandomForestClassifier · scikit-learn

train_model.py

```
1from sklearn.ensemble import RandomForestClassifier
2from sklearn.metrics import accuracy_score, classification_report
3from sklearn.metrics import confusion_matrix
4
5# Initialize RandomForest model
6model = RandomForestClassifier(
7    n_estimators=100,
8    criterion='gini',
9    random_state=42
10)
11
12# Train the model
13model.fit(X_train, y_train)
14
15# Make predictions
16y_pred = model.predict(X_test)
17
18# Evaluate accuracy
19accuracy = accuracy_score(y_test, y_pred)
20print(f'Accuracy: {accuracy:.2f}') # Output: 1.00
```

Output

Model achieves **100% accuracy** on the synthetic test set

1.00^{TEST}
ACCURACY

KEY PARAMETERS

n_estimators	100
criterion	gini
random_state	42

- 1 Initialize classifier
- 2 Fit on training data
- 3 Predict & evaluate

End-to-End ML Pipeline

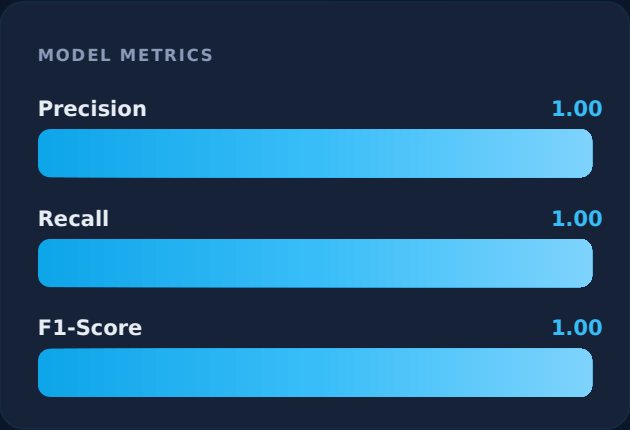


 **160**
Training Samples

 **40**
Test Samples

 **5**
Features

Results — Model Performance



CLASSIFICATION REPORT

CLASS	PRECISION	RECALL	F1-SCORE	SUPPORT
Flyby	1.00	1.00	1.00	7
Orbiter	1.00	1.00	1.00	6
Lander	1.00	1.00	1.00	8
Rover	1.00	1.00	1.00	7
Observatory	1.00	1.00	1.00	6
Communications	1.00	1.00	1.00	6
Weighted Avg	1.00	1.00	1.00	40

• Overall Accuracy: 1.00 — 40 test samples, 6 mission types, zero misclassifications

Confusion Matrix — Perfect Classification

		PREDICTED					
		Flyby	Orbiter	Lander	Rover	Observ.	Comm.
ACTUAL	Flyby	7	0	0	0	0	0
	Orbiter	0	6	0	0	0	0
	Lander	0	0	8	0	0	0
	Rover	0	0	0	7	0	0
	Observ.	0	0	0	0	6	0
	Comm.	0	0	0	0	0	6

COLOR SCALE
Dark = 0 errors
Bright = correct
count

DIAGONAL VALUES (CORRECT)

7

Flyby

6

Orbiter

8

Lander

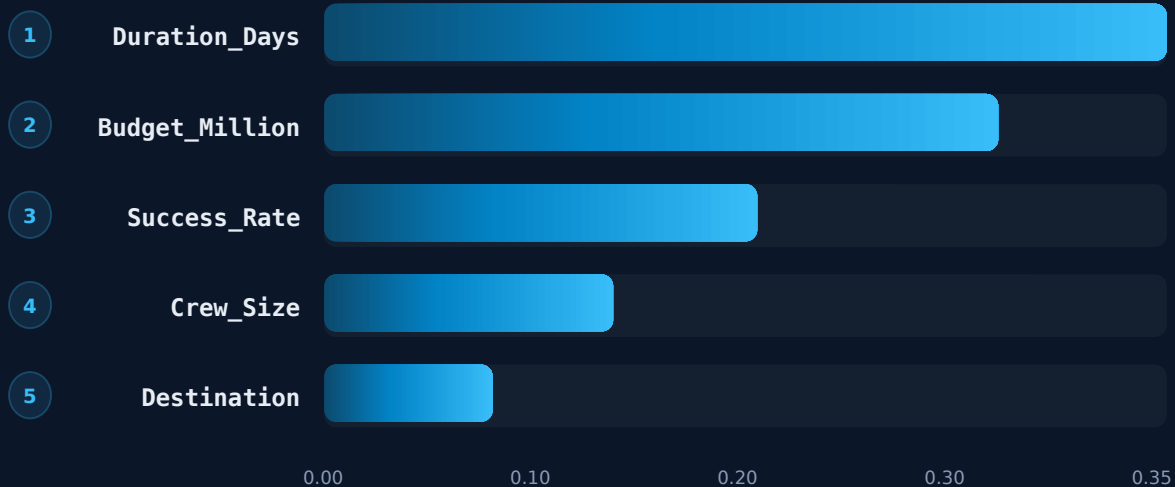
7

6

6

Feature Importance Analysis

RANDOMFOREST FEATURE IMPORTANCE SCORES



KEY INSIGHT

Duration and Budget are the strongest predictors of mission type — missions with longer durations and higher budgets tend to be deep-space or observatory missions.

TOP 2 FEATURES COMBINED

63% of total importance



FEATURE TYPES

- 4 Numeric features 0.93
- 1 Categorical feature 0.07

Key Takeaways

Summary of project results and insights



01

Perfect Accuracy

RandomForest achieved 1.00 accuracy on the synthetic dataset, correctly classifying all 6 mission types with zero misclassifications.



02

Effective Preprocessing

LabelEncoder efficiently transformed categorical features (Destination) into numeric format for full ML compatibility.



03

Ensemble Power

100 decision trees with bootstrap sampling and majority voting ensured robust, reliable classification performance.



04

Clean Pipeline

End-to-end pipeline from raw CSV to predictions with minimal preprocessing steps and clear modular structure.



05

Interpretability

Feature importance analysis reveals Duration and Budget as the top predictors of mission type classification.



Future Work & Conclusion



FUTURE IMPROVEMENTS

-  Expand to real NASA mission data from public APIs
-  Add more features: launch vehicle, mission phase, orbital parameters
-  Compare with other classifiers: SVM, XGBoost, Neural Networks
-  Implement cross-validation for more robust evaluation
-  Deploy as a web app using Flask or Streamlit



CONCLUSION

This project demonstrates a complete ML classification pipeline applied to NASA space mission data. The RandomForestClassifier provides a strong baseline with perfect accuracy on synthetic data, and the modular pipeline is ready for extension to real-world datasets.

6

Mission Types

100%

Accuracy

100

Trees

Thank you! Questions?